

Module: Problem 7

This module provides utility functions for manipulating dictionaries.

Currently it contains a single function, `dict_invert`, which creates an inverted mapping from values to the keys that originally held them.

Functions

`dict_invert(d)`

Description:

Creates an inverted dictionary from the input mapping `d`. For each key-value pair in `d`, the function adds the key to a list associated with the original value in the resulting dictionary. The lists of keys are kept in alphabetical (lexicographic) order.

Parameters: - `d` (*dict*): The dictionary to invert. Keys are expected to be hashable, and values must also be hashable because they become keys in the inverted dictionary.

Returns:

`dict` – An inverted dictionary where each original value maps to a **sorted list** of keys that had that value in `d`.

Examples:

```
# Example with a proper dictionary (expected usage)
input_dict = {'a': 1, 'b': 2, 'c': 1}
result = dict_invert(input_dict)
# result → {1: ['a', 'c'], 2: ['b']}
```

```
# Based on observed runtime behavior (invalid input)
result = dict_invert('hello') # → AttributeError: 'str' object has no attribute 'items'
result = dict_invert(['test']) # → AttributeError: 'list' object has no attribute 'items'
result = dict_invert('')      # → AttributeError: 'str' object has no attribute 'items'
```

Edge Cases / Notes: - The function assumes `d` implements the `items()` method (i.e., is a mapping). Passing a non-dictionary object such as a string or list triggers an `AttributeError`. - If multiple keys share the same value, all those keys will appear in the list for that value, sorted alphabetically. - Values used as new keys must be hashable; unhashable values (e.g., lists) will raise a `TypeError` when used as dictionary keys.